

Technological Institute of the Philippines	Quezon City - Computer Engineering
Course Code:	CPE 027
Code Title:	Digital Signal Processing and Applications
1st Semester	AY 2026-2027
ACTIVITY NO. 2	Signal Operations
Name:	Pascual, Ken Leonard
Section:	CPE41S2
Date Performed:	17 July 2026
Date Submitted:	17 July 2026
Instructor:	Engr. Jimlord M. Quejado

1. Objectives

The aim of this experiment is to generate sinusoidal signal, and perform the basic signal operations such as time shifting, time reversal, time scaling, amplitude scaling, signal addition and multiplication. This includes programmatically visualizing, calculating, and storing, using a high-level language.

2. Intended Learning Outcomes (ILOs)

After this activity, the students should be able to:

- Develop a program that creates signals, and performs and visualizes the signal operations.

3. Discussion

Signals constitute an important part of our daily life. Anything that carries some information is called a signal. A signal is defined as a single-valued function of one or more independent variables which contain some information. A signal is also defined as a physical quantity that varies with time, space or any other independent variable. A signal may be represented in time domain or frequency domain. Human speech is a familiar example of a signal. Electric current and voltage are also examples of signals. A signal can be a function of one or more independent variables. A signal may be a function of time, temperature, position, pressure, distance etc. If a signal depends on only one independent variable, it is called a one- dimensional signal, and if a signal depends on two independent variables, it is called a two- dimensional signal.

When we process a signal, this signal may undergo several manipulations involving the independent variable or the amplitude of the signal. The basic operations on sequences are as follows:

1. Time shifting
2. Time reversal
3. Time scaling
4. Amplitude scaling
5. Signal addition
6. Signal multiplication

The first three operations correspond to transformation in independent variable n of a signal. The last three operations correspond to transformation on amplitude of a signal.

4. Resources

The activity will require the following software, tools and equipment:

- IDE or any online code editor

5. Directions:

1. Using NumPy libraries, create a sinusoidal signal using the following formula:

$$x_1(t) = A \sin(2\pi f t + \phi)$$

- where A represents the amplitude of the signal, f denotes the frequency of the signal and ϕ indicates the phase of the signal.
- Set $A = 3.2$, $f = 2.4$, and $\phi = \frac{3\pi}{4}$

```
In [14]: import numpy as np
import matplotlib.pyplot as plt

# Parameters
A = 3.2 # amplitude / height of the wave
f = 2.4 # frequency
phi = 3 * np.pi / 4 # phase / shift; slides the wave left or right

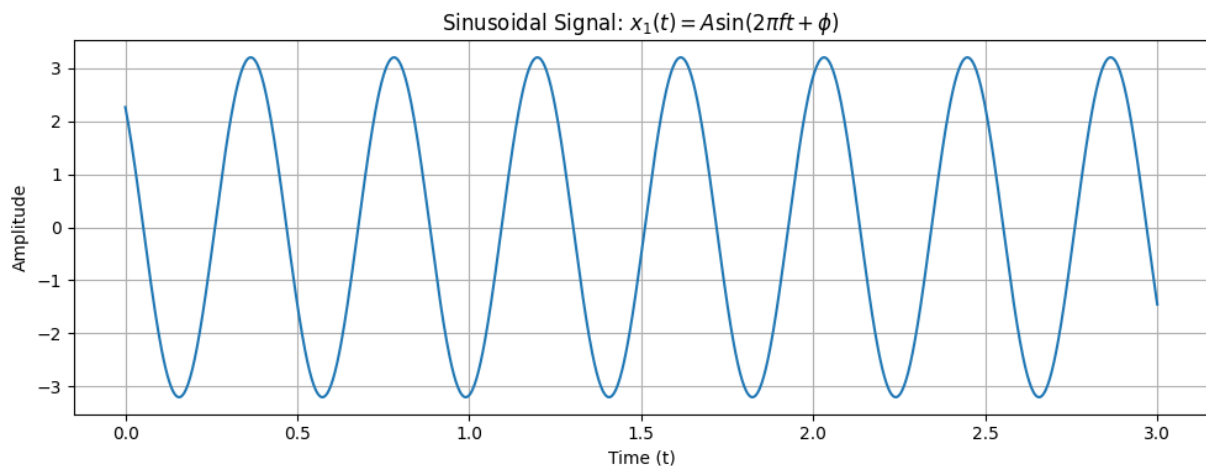
# Time vector
t = np.linspace(0, 3, 1000)

# Generate sinusoidal signal
x1 = A * np.sin(2 * np.pi * f * t + phi)

# Plot
```

```
plt.figure(figsize=(10, 4))
plt.plot(t, x1)
plt.title(r'Sinusoidal Signal:  $x_1(t) = A\sin(2\pi f t + \phi)$ ')
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.tight_layout()
plt.show()

print(f'Amplitude: {A}, Frequency: {f} Hz, Phase: {np.degrees(phi):.1f} degrees')
print(f'Signal length: {len(x1)} samples over {t[-1]:.1f} seconds')
```



Amplitude: 3.2, Frequency: 2.4 Hz, Phase: 135.0 degrees
Signal length: 1000 samples over 3.0 seconds

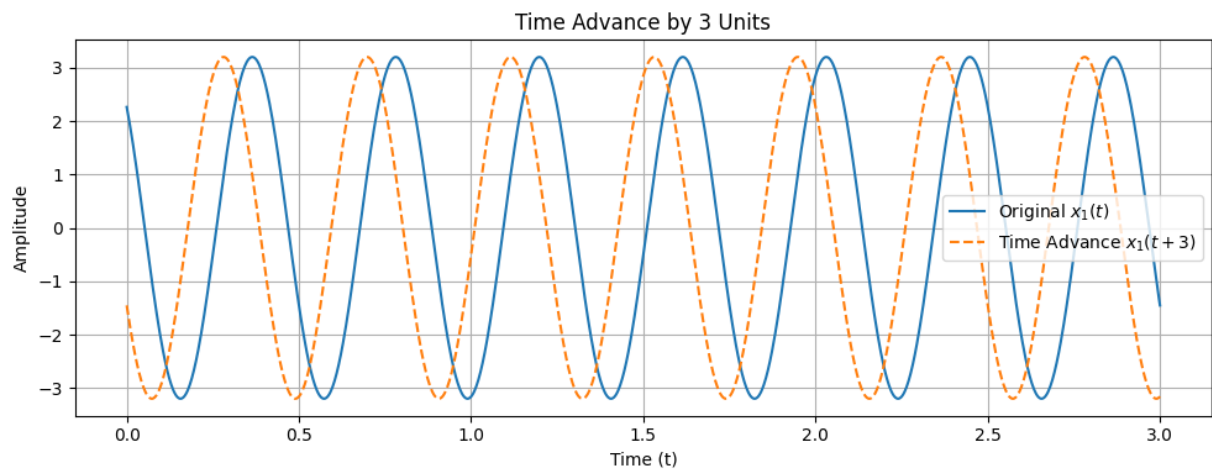
As seen from the code and graph, the values of A, f, and phi directly affect the amplitude, frequency, and phase shift, respectively.

Amplitude is the height of the wave, frequency is how long the wave pattern is in terms of the x-axis (time), and the phase shift determines the starting location of the wave in terms of the x-axis (time)

- Using the created sinusoidal signal, perform time advance with 3 units. Then graph and compare the result signal to original sinusoidal signal.

```
In [24]: # Time advance: x1(t + 3) means evaluate x1 at (t + 3)
t_shifted = t + 3
x1_advance = A * np.sin(2 * np.pi * f * t_shifted + phi) # use t+3

plt.figure(figsize=(10, 4))
plt.plot(t, x1, label='Original  $x_1(t)$ ')
plt.plot(t, x1_advance, '--', label='Time Advance  $x_1(t+3)$ ')
plt.title('Time Advance by 3 Units')
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



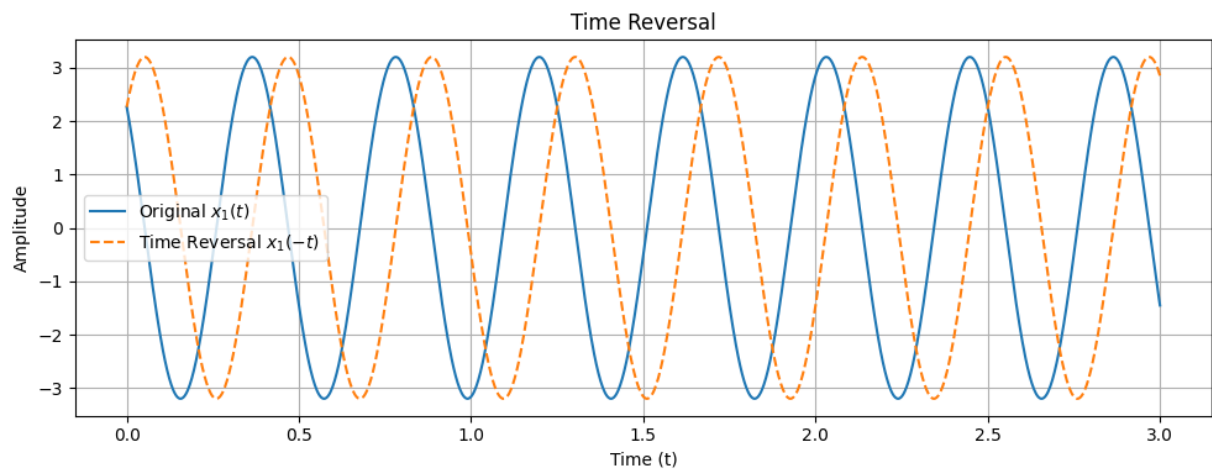
Time advance shifts the signal to the left. As seen from the graph, it means that the advanced signal reaches its peaks/valleys earlier than the original. Interestingly, to do this, the original wave is evaluated at an earlier time, hence the $(t+3)$ in the code.

Time advance basically answers the question "what does the wave do 3 units in its own future?"

3. Perform time reversal with the created sinusoidal signal. Then graph and compare the result signal to original sinusoidal signal.

```
In [7]: # Time reversal: x1(-t) flips the signal about the vertical axis
x1_reversed = A * np.sin(2 * np.pi * f * (-t) + phi) #-t instead of t

# Plot comparison
plt.figure(figsize=(10, 4))
plt.plot(t, x1, label='Original  $x_1(t)$ ')
plt.plot(t, x1_reversed, '--', label='Time Reversal  $x_1(-t)$ ')
plt.title('Time Reversal')
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

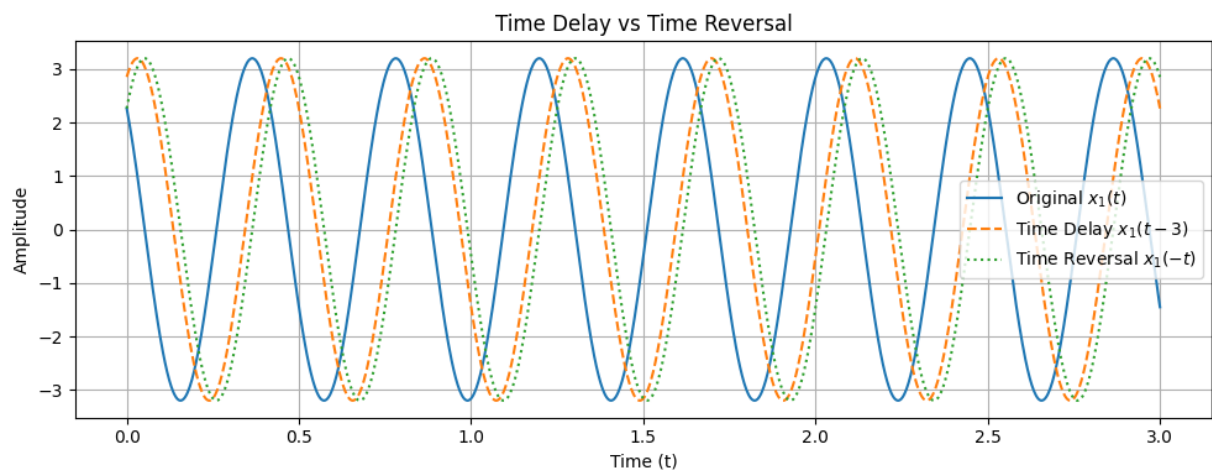


Time reversal mirrors the signal across the vertical (amplitude) axis. Events that occurred early in the original now appear late, and vice versa.

Time reversal mirrors the signal across the vertical (amplitude) axis. Visually, it results to events that occurred early in the original wave to now appear late, and vice versa. This is different from just doing a time delay ($t-3$), as seen from the graph below.

```
In [25]: # Time delay:  $x_1(t-3)$  shifts the signal to the right by 3 units
x1_delayed = A * np.sin(2 * np.pi * f * (t - 3) + phi)

# Plot comparison: Time Delay vs Time Reversal
plt.figure(figsize=(10, 4))
plt.plot(t, x1, label='Original  $x_1(t)$ ', linewidth=1.5)
plt.plot(t, x1_delayed, '--', label='Time Delay  $x_1(t-3)$ ')
plt.plot(t, x1_reversed, ':', label='Time Reversal  $x_1(-t)$ ')
plt.title('Time Delay vs Time Reversal')
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

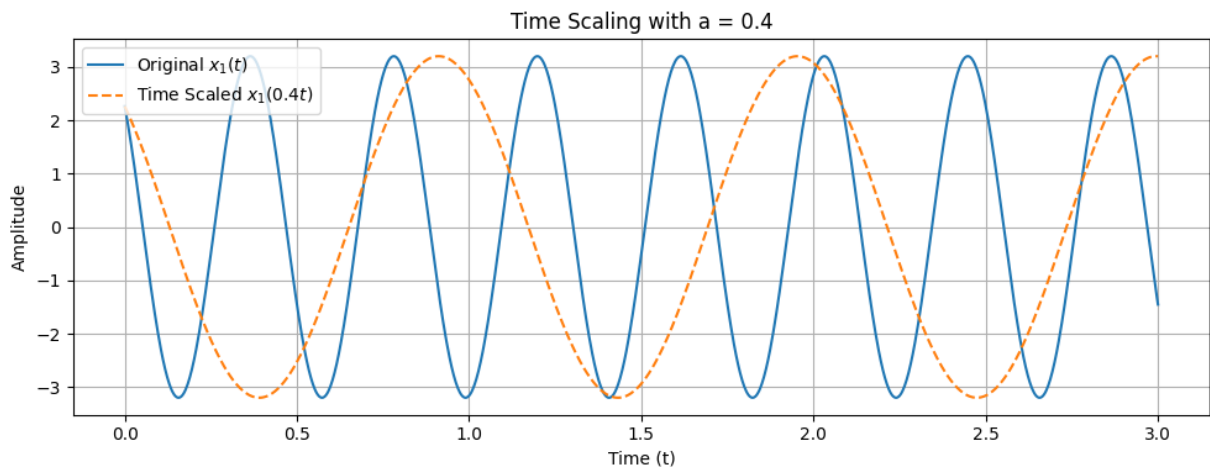


4. Perform time scaling using $a=0.4$ with the created sinusoidal signal. Then graph and compare the result signal to original sinusoidal signal.

```
In [21]: # Time scaling:  $x_1(a*t)$  with  $a = 0.4$  compresses/stretches the signal
# Since  $a < 1$ , the signal is stretched (period increases)
a_ts = 0.4
x1_scaled = A * np.sin(2 * np.pi * f * (a_ts * t) + phi)

# Plot comparison
plt.figure(figsize=(10, 4))
plt.plot(t, x1, label='Original  $x_1(t)$ ')
plt.plot(t, x1_scaled, '--', label=f'Time Scaled  $x_1({a\_ts}t)$ ')
plt.title(f'Time Scaling with  $a = {a\_ts}$ ')
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print(f'Time scaling by  $a = {a\_ts}$  ( $< 1$ ) stretches the signal.')
print(f'The effective frequency becomes  $f*a = {:.2f}$  Hz, so the period increa
```



Time scaling by $a = 0.4$ (< 1) stretches the signal.
 The effective frequency becomes $f*a = 0.96$ Hz, so the period increases.

As observed from the graph, $a=0.4$ changes how fast the signal "plays" through time relative to the original, compressing or stretching it horizontally.

Visually, this results to a wave that looks wider.

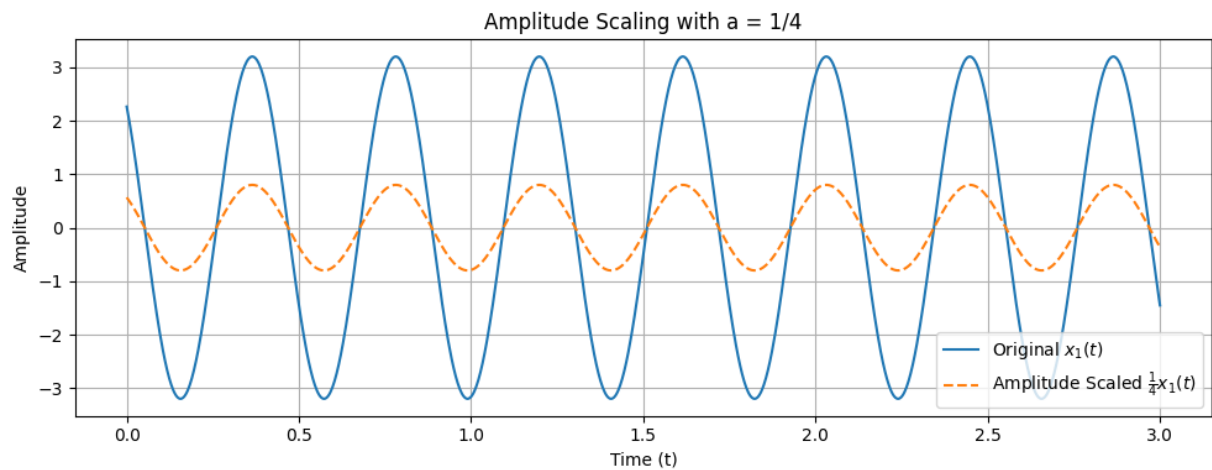
5. Perform amplitude scaling using $a=1/4$ with the created sinusoidal signal. Then graph and compare the result signal to original sinusoidal signal.

```
In [9]: a_amp = 1/4
x1_amp_scaled = a_amp * x1

# Plot comparison
```

```
plt.figure(figsize=(10, 4))
plt.plot(t, x1, label='Original  $x_1(t)$ ')
plt.plot(t, x1_amp_scaled, '--', label=rf'Amplitude Scaled  $\frac{1}{4}x_1(t)$ ')
plt.title('Amplitude Scaling with a = 1/4')
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print(f'Original amplitude: {A}')
print(f'Scaled amplitude: {A * a_amp}')
print('Amplitude scaling only affects the vertical magnitude; frequency and
```



Original amplitude: 3.2

Scaled amplitude: 0.8

Amplitude scaling only affects the vertical magnitude; frequency and phase remain unchanged.

In amplitude scaling, we just multiply the whole output by a constant, thus the $a_{amp} * x_1$

Visually, this makes the wave reach a lower amplitude. In practice, this is like adjusting the volume knob.

6. Create a separate exponential signal using the following formula:

$$x_2(t) = a^t$$

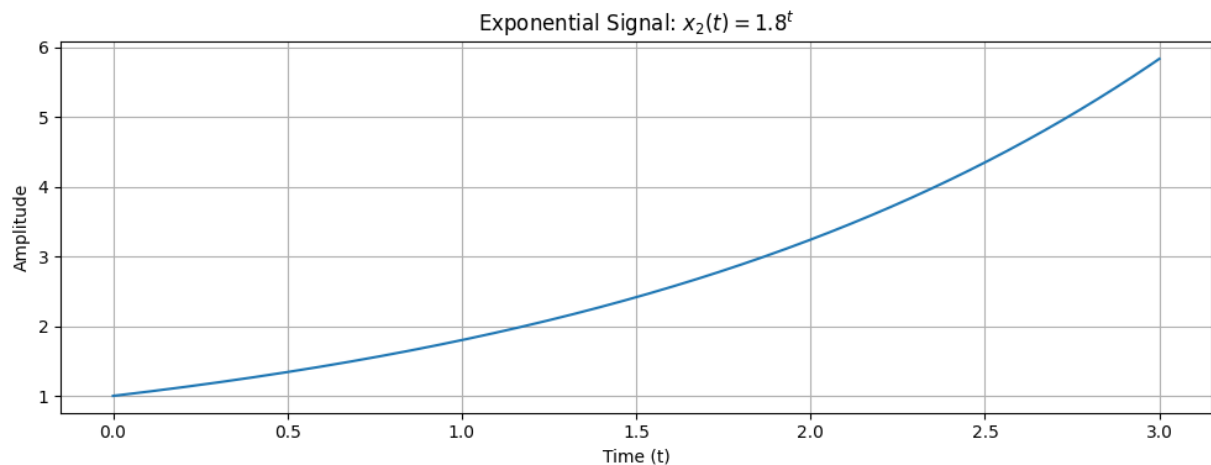
- Graph the result signal where $a = 1.8$

```
In [10]: # Exponential signal:  $x_2(t) = a^t$  with  $a = 1.8$ 
a_exp = 1.8
x2 = a_exp ** t

# Plot
plt.figure(figsize=(10, 4))
plt.plot(t, x2)
plt.title(r'Exponential Signal:  $x_2(t) = 1.8^t$ ')
```

```
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.tight_layout()
plt.show()

print(f'Base a = {a_exp} (> 1), so the signal grows exponentially over time.
print(f'At t=0, x2(0) = {a_exp**0}, at t=1, x2(1) = {a_exp**1:.1f}, at t=3,
```



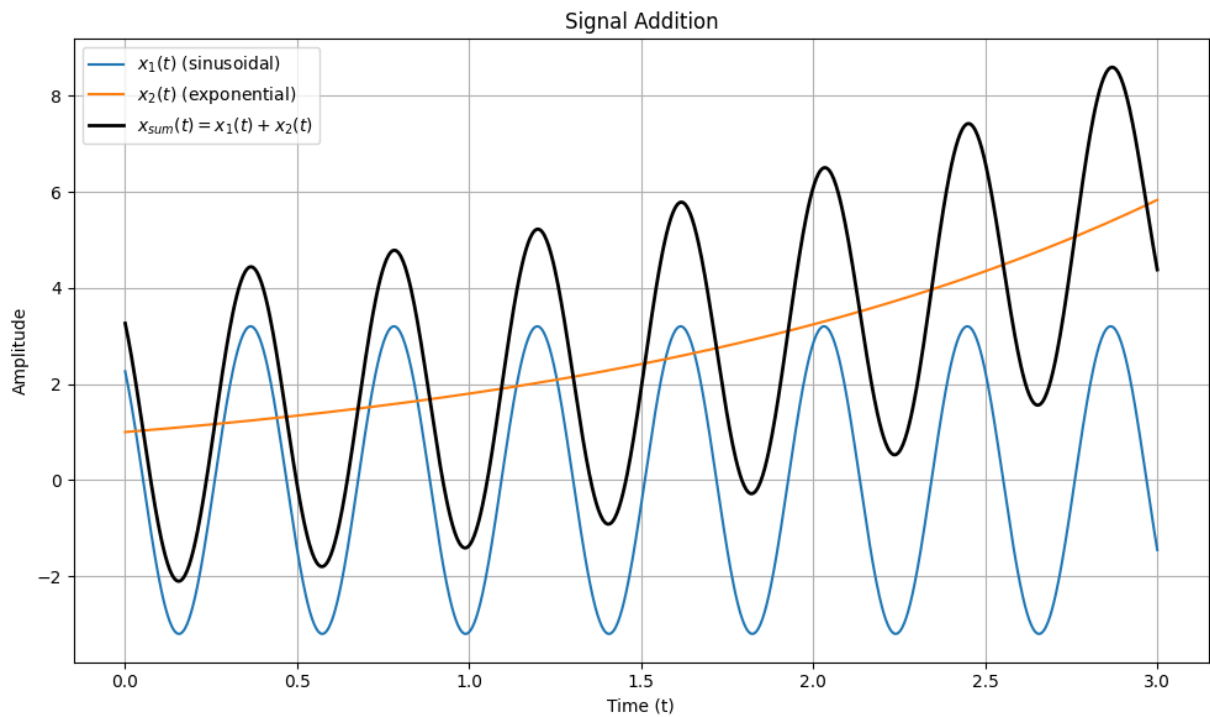
Base $a = 1.8$ (> 1), so the signal grows exponentially over time.
 At $t=0$, $x_2(0) = 1.0$, at $t=1$, $x_2(1) = 1.8$, at $t=3$, $x_2(3) = 5.83$

7. Perform signal addition for x_1 and x_2 and plot the result.

```
In [11]: # Signal addition: x_sum(t) = x1(t) + x2(t)
# Both signals must have the same time vector
x_sum = x1 + x2

# Plot all three signals
plt.figure(figsize=(10, 6))
plt.plot(t, x1, label='$x_1(t)$ (sinusoidal)')
plt.plot(t, x2, label='$x_2(t)$ (exponential)')
plt.plot(t, x_sum, 'k-', linewidth=2, label='$x_{sum}(t) = x_1(t) + x_2(t)$')
plt.title('Signal Addition')
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print('The sum combines the oscillating sinusoid with the growing exponential')
print('The exponential component dominates at larger t values.')
```

The sum combines the oscillating sinusoid with the growing exponential. The exponential component dominates at larger t values.

8. Perform signal multiplication for x_1 and x_2 . Then graph and compare the product signal to summed signal.

```
In [12]: # Signal multiplication:  $x_{prod}(t) = x_1(t) * x_2(t)$ 
x_prod = x1 * x2

# Plot comparison: product vs sum
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

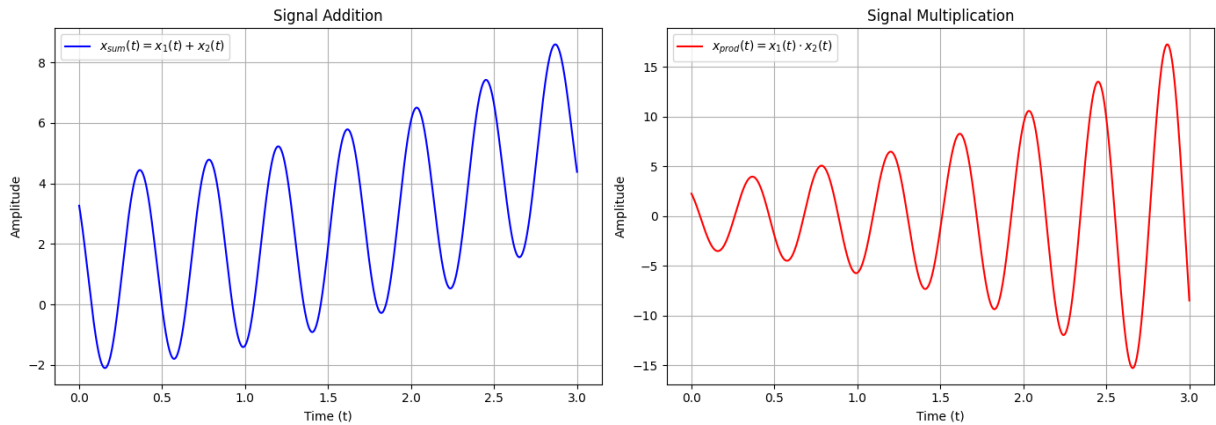
axes[0].plot(t, x_sum, 'b-', label='$x_{sum}(t) = x_1(t) + x_2(t)$')
axes[0].set_title('Signal Addition')
axes[0].set_xlabel('Time (t)')
axes[0].set_ylabel('Amplitude')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(t, x_prod, 'r-', label='$x_{prod}(t) = x_1(t) \cdot x_2(t)$')
axes[1].set_title('Signal Multiplication')
axes[1].set_xlabel('Time (t)')
axes[1].set_ylabel('Amplitude')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()

print('In multiplication, the exponential envelope modulates the sinusoidal')
print('The sinusoidal oscillations grow in amplitude as t increases due to t')
print('In addition, the sinusoid rides on top of the exponential curve.')
```

```
<>:14: SyntaxWarning: invalid escape sequence '\c'
<>:14: SyntaxWarning: invalid escape sequence '\c'
/tmp/ipykernel_1105/178614301.py:14: SyntaxWarning: invalid escape sequence
'\c'
axes[1].plot(t, x_prod, 'r-', label='$x_{prod}(t) = x_1(t) \cdot x_2(t)$')
```



In multiplication, the exponential envelope modulates the sinusoidal amplitude.

The sinusoidal oscillations grow in amplitude as t increases due to the exponential factor.

In addition, the sinusoid rides on top of the exponential curve.

6. Conclusion

In this activity, I was able to develop a program that creates signals, and performs and visualizes the signal operations. I was also able to learn about the following concepts in signal processing:

- Time advance — shifting a signal earlier in time, $x(t+a)$
- Time reversal — flipping a signal about the vertical axis, $x(-t)$
- Time scaling — compressing or stretching a signal along the time axis, $x(at)$
- Amplitude scaling — multiplying the signal's values by a constant, $a \cdot x(t)$
- Signal addition — adding two signals point-by-point
- Signal multiplication — multiplying two signals point-by-point

7. Artificial Intelligence (AI) Use Disclosure Statement

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Claude, Opencode

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc

Purpose of Use: Code assistance and debugging; concept summarization

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc

Extent of Use: AI was used to summarize concepts and understand the concepts involved. In the events that the code snippets didn't work, AI was used for code debugging.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.